Name:G SHIVANI

- Roll no : 2520090046

SKILL WEEK-1

Session -1

Task1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

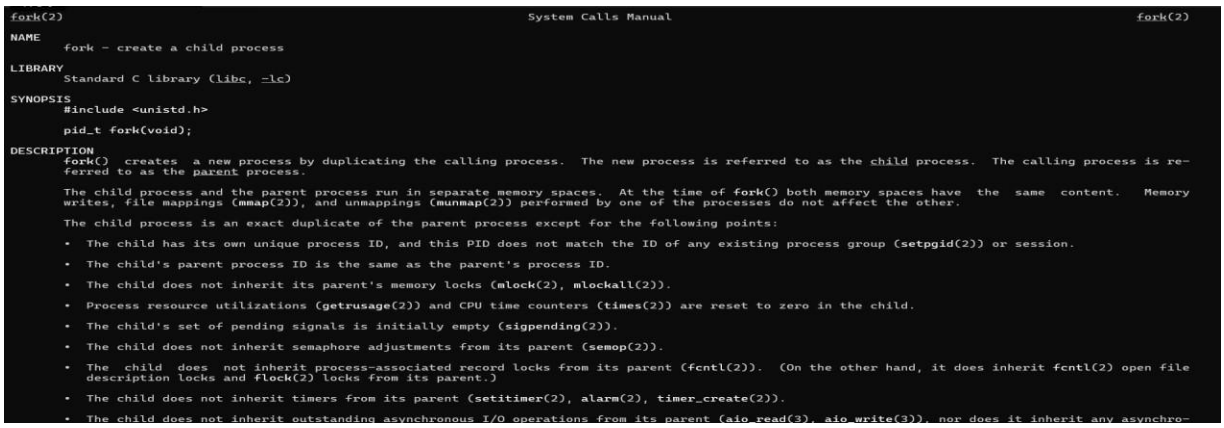
```
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

```
root@Shivani:~# mkdir week2
root@Shivani:~# cd week2
root@Shivani:~/week2# mkdir task2
Command 'mkdir' not found, did you mean:
  command 'mkdir' from deb coreutils (9.4-3ubuntu6.2)
Try: apt install <deb name>
root@Shivani:~/week2# cd week2
-bash: cd: week2: No such file or directory
root@Shivani:~/week2# mkdir src  include obj bin docs tests scripts assets
root@Shivani:~/week2# touch .gitignore README.md LICENSE Makefile
root@Shivani:~/week2# touch src/main.c
touch src/shell.c
touch src/parser.c
touch src/executor.c
touch src/builtin.c
root@Shivani:~/week2# touch include/shell.h
touch include/parser.h
touch include/executor.h
touch include/builtin.h
root@Shivani:~/week2# touch bin/my_shell
root@Shivani:~/week2# touch docs/design.md
touch docs/report.pdf
root@Shivani:~/week2# touch tests/test_parser.c
touch tests/test_executor.c
root@Shivani:~/week2# touch scripts/run.sh
root@Shivani:~/week2# touch assets/architecture.png
root@Shivani:~/week2# sudo apt update
sudo apt install tree
```

Observation: I explored the Linux development environment and practiced organizing a shell project into appropriate directories and files. I also learned the

role of GCC in compiling C programs and understood how Git helps in maintaining and tracking the project files. This activity helped me develop a clear understanding of how a software project is structured and managed systematically.

Task2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, and practice system call tracing.



fork(2) System Calls Manual fork(2)

NAME
fork - create a child process

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
#include <unistd.h>

pid_t fork(void);

DESCRIPTION
fork() creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of fork() both memory spaces have the same content. Memory writes, file mappings (mmap(2)), and unmappings (munmap(2)) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- The child has its own unique process ID, and this PID does not match the ID of any existing process group (setpgid(2)) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks (mlock(2), mlockall(2)).
- Process resource utilizations (getrusage(2)) and CPU time counters (times(2)) are reset to zero in the child.
- The child's set of pending signals is initially empty (sigpending(2)).
- The child does not inherit semaphore adjustments from its parent (semop(2)).
- The child does not inherit process-associated record locks from its parent (fcntl(2)). (On the other hand, it does inherit fcntl(2) open file description locks and flock(2) locks from its parent.)
- The child does not inherit timers from its parent (setitimer(2), alarm(2), timer_create(2)).
- The child does not inherit outstanding asynchronous I/O operations from its parent (aio_read(3), aio_write(3)), nor does it inherit any asynchro-

- The child does not inherit directory change notifications (dnotify) from its parent (see the description of F_NOTIFY in fcntl(2)).
- The prctl(2) PR_SET_PDEATHSIG setting is reset so that the child does not receive a signal when its parent terminates.
- The default timer slack value is set to the parent's current timer slack value. See the description of PR_SET_TIMERSLACK in prctl(2).
- Memory mappings that have been marked with the madvise(2) MADV_DONTFORK flag are not inherited across a fork().
- Memory in address ranges that have been marked with the madvise(2) MADV_WIPEONFORK flag is zeroed in the child after a fork(). (The MADV_WIPEONFORK setting remains in place for those address ranges in the child.)
- The termination signal of the child is always SIGCHLD (see clone(2)).
- The port access permission bits set by ioperm(2) are not inherited by the child; the child must turn on any bits that it requires using ioperm(2).

Note the following further points:

- The child process is created with a single thread—the one that called fork(). The entire virtual address space of the parent is replicated in the child, including the states of mutexes, condition variables, and other pthreads objects; the use of pthread_atfork(3) may be helpful for dealing with problems that this can cause.
- After a fork() in a multithreaded program, the child can safely call only async-signal-safe functions (see signal-safety(7)) until such time as it calls execve(2).
- The child inherits copies of the parent's set of open file descriptors. Each file descriptor in the child refers to the same open file description (see open(2)) as the corresponding file descriptor in the parent. This means that the two file descriptors share open file status flags, file offset, and signal-driven I/O attributes (see the description of F_SETOWN and F_SETSIG in fcntl(2)).
- The child inherits copies of the parent's set of open message queue descriptors (see mq_overview(7)). Each file descriptor in the child refers to the same open message queue description as the corresponding file descriptor in the parent. This means that the two file descriptors share the same flags (mq_flags).
- The child inherits copies of the parent's set of open directory streams (see opendir(3)). POSIX.1 says that the corresponding directory streams in the parent and child may share the directory stream positioning; on Linux/glibc they do not.

RETURN VALUE

On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and errno is set to indicate the error.

EAGAIN The caller is operating under the SCHED_DEADLINE scheduling policy and does not have the reset-on-fork flag set. See sched(7).

ENOMEM fork() failed to allocate the necessary kernel structures because memory is tight.

ENOMEM An attempt was made to create a child process in a PID namespace whose "init" process has terminated. See pid_namespaces(7).

ENOSYS fork() is not supported on this platform (for example, hardware without a Memory-Management Unit).

ERESTARTNOINTR (since Linux 2.6.17)
System call was interrupted by a signal and will be restarted. (This can be seen only during a trace.)

VERSIONS

C library/kernel differences

Since glibc 2.3.3, rather than invoking the kernel's fork() system call, the glibc fork() wrapper that is provided as part of the NPTL threading implementation invokes clone(2) with flags that provide the same effect as the traditional system call. (A call to fork() is equivalent to a call to clone(2) specifying flags as just SIGCHLD.) The glibc wrapper invokes any fork handlers that have been established using pthread_atfork(3).

Async-signal safety

_Fork(3) is an async-signal safe variant of fork(2).

STANDARDS

POSIX.1-2024.

HISTORY

4.3BSD, SVr4, POSIX.1-1988.

NOTES

Under Linux, fork() is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables, and to create a unique task structure for the child.

EXAMPLES

See pipe(2) and wait(2) for more examples.

```
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int
main(void)
{
    pid_t pid;

    if (signal(SIGCHLD, SIG_IGN) == SIG_ERR) {
        perror("signal");
        exit(EXIT_FAILURE);
    }
    pid = fork();
    switch (pid) {
    case -1:
        perror("fork");
        exit(EXIT_FAILURE);
    case 0:
        puts("Child exiting.");
        fflush(stdout);
        _exit(EXIT_SUCCESS);
    default:
        printf("Child is PID %jd\n", (intmax_t) pid);
        puts("Parent exiting.");
        exit(EXIT_SUCCESS);
    }
}
```

SEE ALSO

clone(2), execve(2), exit(2), _exit(2), setrlimit(2), unshare(2), vfork(2), wait(2), daemon(3), _Fork(3), pthread_atfork(3), capabilities(7), cre-

NAME

execl, execlp, execle, execv, execvp, execvpe - execute a file

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <unistd.h>

extern char **environ;

int execl(const char *path, const char *arg, ...
                /*, (char *) NULL */);
int execlp(const char *file, const char *arg, ...
                /*, (char *) NULL */);
int execle(const char *path, const char *arg, ...
                /*, (char *) NULL, char *const envp[] */);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

execvpe():
_GNU_SOURCE

DESCRIPTION

The **exec**() family of functions replaces the current process image with a new process image. The functions described in this manual page are layered on top of **execve**(2). (See the manual page for **execve**(2) for further details about the replacement of the current process image.)

The initial argument for these functions is the name of a file that is to be executed.

The functions can be grouped based on the letters following the "exec" prefix.

l - execl(), execlp(), execle()
The const char *arg and subsequent ellipses can be thought of as arg0, arg1, ..., argn. Together they describe a list of one or more pointers to null-terminated strings that represent the argument list available to the executed program. The first argument, by convention, should point to the filename associated with the file being executed. The list of arguments must be terminated by a null pointer, and, since these are variadic func-

By contrast with the 'l' functions, the 'v' functions (below) specify the command-line arguments of the executed program as a vector.

v - execv(), execvp(), execvpe()
The char *const argv[] argument is an array of pointers to null-terminated strings that represent the argument list available to the new program. The first argument, by convention, should point to the filename associated with the file being executed. The array of pointers must be terminated by a null pointer.

e - execle(), execvpe()
The environment of the new process image is specified via the argument envp. The envp argument is an array of pointers to null-terminated strings and must be terminated by a null pointer.

All other **exec**() functions (which do not include 'e' in the suffix) take the environment for the new process image from the external variable environ in the calling process.

p - execlp(), execvp(), execvpe()
These functions duplicate the actions of the shell in searching for an executable file if the specified filename does not contain a slash (/) character. The file is sought in the colon-separated list of directory pathnames specified in the **PATH** environment variable. If this variable isn't defined, the path list defaults to a list that includes the directories returned by confstr(_CS_PATH) (which typically returns the value "/bin:/usr/bin") and possibly also the current working directory; see VERSIONS for further details.

execvpe() searches for the program using the value of **PATH** from the caller's environment, not from the envp argument.

If the specified filename includes a slash character, then **PATH** is ignored, and the file at the specified pathname is executed.

In addition, certain errors are treated specially.

If permission is denied for a file (the attempted **execve**(2) failed with the error **EACCES**), these functions will continue searching the rest of the search path. If no other file is found, however, they will return with errno set to **EACCES**.

If the header of a file isn't recognized (the attempted **execve**(2) failed with the error **ENOEXEC**), these functions will execute the shell (/bin/sh) with the path of the file as its first argument. (If this attempt fails, no further searching is done.)

All other **exec**() functions (which do not include 'p' in the suffix) take as their first argument a (relative or absolute) pathname that identifies the program to be executed.

RETURN VALUE

The **exec**() functions return only if an error has occurred. The return value is -1, and errno is set to indicate the error.

ATTRIBUTES

For an explanation of the terms used in this section, see **attributes**(7).

Interface	Attribute	Value
execl(), execle(), execv()	Thread safety	MT-Safe
execlp(), execvp(), execvpe()	Thread safety	MT-Safe env

VERSIONS

The default search path (used when the environment does not contain the variable **PATH**) shows some variation across systems. It generally includes /bin and /usr/bin (in that order) and may also include the current working directory. On some other systems, the current working is included after /bin and /usr/bin, as an anti-Trojan-horse measure. The glibc implementation long followed the traditional default where the current working directory is included at the start of the search path. However, some code refactoring during the development of glibc 2.24 caused the current working directory to be dropped altogether from the default search path. This accidental behavior change is considered mildly beneficial, and won't be reverted.

The behavior of **execlp**() and **execvp**() when errors occur while attempting to execute the file is historic practice, but has not traditionally been documented and is not specified by the POSIX standard. BSD (and possibly other systems) do an automatic sleep and retry if **ETXTBSY** is encountered. Linux treats it as a hard error and returns immediately.

Traditionally, the functions **execlp**() and **execvp**() ignored all errors except for the ones described above and **ENOMEM** and **E2BIG**, upon which they returned. They now return if any error other than the ones described above occurs.

STANDARDS

environ
execl()
execlp()
execle()
execv()
execvp()
POSIX.1-2008.

execvpe()
GNU.

ATTRIBUTES

For an explanation of the terms used in this section, see **attributes**(7).

Interface	Attribute	Value
execl(), **execle**(), **execv**()	Thread safety	MT-Safe
execlp(), **execvp**(), **execvpe**()	Thread safety	MT-Safe env

VERSIONS

The default search path (used when the environment does not contain the variable **PATH**) shows some variation across systems. It generally includes /bin and /usr/bin (in that order) and may also include the current working directory. On some other systems, the current working is included after /bin and /usr/bin, as an anti-Trojan-horse measure. The glibc implementation long followed the traditional default where the current working directory is included at the start of the search path. However, some code refactoring during the development of glibc 2.24 caused the current working directory to be dropped altogether from the default search path. This accidental behavior change is considered mildly beneficial, and won't be reverted.

The behavior of **execlp**() and **execvp**() when errors occur while attempting to execute the file is historic practice, but has not traditionally been documented and is not specified by the POSIX standard. BSD (and possibly other systems) do an automatic sleep and retry if **ETXTBSY** is encountered. Linux treats it as a hard error and returns immediately.

Traditionally, the functions **execlp**() and **execvp**() ignored all errors except for the ones described above and **ENOMEM** and **E2BIG**, upon which they returned. They now return if any error other than the ones described above occurs.

STANDARDS

environ
execl()
execlp()
execle()
execv()
execvp()
POSIX.1-2008.

execvpe()
GNU.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent Process: PID = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d\n", getpid());
        printf("Child's Parent PID = %d\n", getppid());

        printf("Child is executing 'ls' using exec()...\n");

        execlp("ls", "ls", "-l", NULL);

        // This executes only if exec() fails
        perror("exec failed");
    }
    else {
        // Parent process
        printf("Parent created Child with PID = %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }
```

```
Parent Process: PID = 13422
Parent created Child with PID = 13423
Child Process: PID = 13423
Child's Parent PID = 13422
Child is executing 'ls' using exec()...
total 20
-rwxr-xr-x 1 madhulika madhulika 16256 Aug  7 16:36 process_demo
-rw-r--r-- 1 madhulika madhulika   770 Aug  7 16:36 process_demo.c
Child process completed.
```

Observation: I gained practical knowledge of process management in the Linux operating system by studying process abstraction and the creation of new processes using the fork() system call. I understood how a parent process creates and manages a child process and how the exec() family of functions is used to replace the child process with a new program. I also inspected the process hierarchy using process-tree commands and used system-call tracing to observe the system calls performed during program execution. This practical helped me understand the working relationship between processes and how the operating system manages process creation, execution, and communication

Session -2

Task1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, and Test Interactive Loop.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        // Display prompt
        printf("myshell> ");
        fflush(stdout);

        // Read user input
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Display entered command
        printf("You entered: %s\n", input);
    }

    return 0;
}
```

```
myshell> hello
You entered: hello
myshell> text
You entered: text
myshell> ls
You entered: ls
myshell> exit
Exiting shell...
```

Observation: I learned how to develop and test an interactive command loop for a basic Linux shell. I understood how to display a prompt, accept and process user input, and continuously repeat the loop until an exit condition is encountered. I also gained practical knowledge of using conditional statements and loops to control program execution. This activity helped me understand the basic control flow of an interactive shell and how user commands are handled efficiently.

Task 2: Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define BUFFER_SIZE 100

int main() {
    struct termios oldt, newt;
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    // Get current terminal settings
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    // Disable canonical mode
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("myshell> ");
    fflush(stdout);

    while (1) {
        // Capture keyboard input
        ch = getchar();

        // Handle Enter key
        if (ch == '\n') {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            // Exit condition
            if (strcmp(buffer, "exit") == 0) {
                break;
            }

            index = 0;
            printf("myshell> ");
            fflush(stdout);
        }

        // Handle Backspace
```

```c
        // Handle Backspace
        else if (ch == 127 || ch == '\b') {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }

        // Handle normal characters
        else if (index < BUFFER_SIZE - 1) {
            buffer[index++] = ch;
            putchar(ch);
            fflush(stdout);
        }
    }

    // Restore terminal settings
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    printf("Exiting shell...\n");

    return 0;
}
```

```
myshell> hello
You entered: hello
myshell> hello world
You entered: hello world
myshell> helllo
You entered: helllo
myshell> exit
You entered: exit
Exiting shell
```

Observation: I learned how to capture and process keyboard input in an interactive shell environment using an input buffer. I understood how individual characters are stored and processed, how the Backspace key can be used to remove characters, and how the Enter key is used to submit the entered command. I also practiced handling multi-character commands and managing the input buffer efficiently. This task helped me understand how a command-line interface handles user interaction at the character level and how different keyboard inputs are processed within a shell program.